

CMPSC 448
-- Project Report --

Identifying LLMs from Their Responses

Team Leader: *Raj Mamidala*
Members: *Kenny Liang, Aditya Raj*

CMPSC 448 - Machine Learning and AI
Fall Semester, 2026

GitHub Link: https://github.com/RM-28/Identifying-LLMs-from-Their-Response

Problem Statement

LLMs such as GPT, Claude, and LLama-based chatbots produce much of the text that we read online, which raises the question of who actually wrote the text. Suppose a piece of text given, is it possible to tell which AI model wrote it? Different models are trained with different methods, so they may leave distinct linguistic, stylistic, and structural "fingerprints" in their responses, such as how long they write, which words and phrases they favor and how they format the answers.

In this project, this is a multi-class text classification problem. Each example is a triple (LLM_name, LLM_input, LLM_output), which has the name of the model, the user's prompt, and the model's response. The idea is given the text, the goal is to predict which of the four LLMs produced it: GPT-3.5-turbo, Claude-v1, PaLM-2, and Vicuna-13B.

We build and compare three neural network classifiers, a CNN, a RNN, and a hybrid of the two and use them to answer these research questions:

- RQ1: Can we identify which LLM generated a response?
- RQ2: Does the user's prompt help identify the LLM?
- RQ3: Do LLM fingerprints generalize across tasks or domains?
- RQ4: What characteristics distinguish different LLMs?

Dataset and Processing

https://huggingface.co/datasets/lmsys/chatbot_arena_conversations

Our dataset is taken directly from Lmsys's Chatbot Arena Conversations Dataset, a public dataset that we queried from Huggingface. This dataset consists of "battles" that took place where two LLM's were given a prompt and users decided on which response was better. Our preprocessing step narrowed down the model count to just four, "gpt-3.5-turbo", "claude-v1", "vicuna-13b", and "palm-2". Along with cleaning the data, we ensured that the number of datapoints for each model were equal before creating our train/val/test split of 70/15/15 respectively.

The dataset is tokenized with NLTK and is cut down to 256 tokens. The vocabulary is built from the training set only, keeping words that appear at least twice. All other words map to <unk> token, and padding uses index 0. These tokens are mapped to pretrained 300-dimensional fastText vectors which are fine-tuned during training.

CNN Implementation

A Convolutional Neural Network uses a filter to scan through the given input and learn patterns. Our implementation features filter_sizes of [3, 4, 5], each having 100 filters, adding up to 300 filters in total. Our dropout is 0.5 (50%), representing how often we turn off connections in our network, which is crucial to preventing the model from memorizing training data (overfitting). We also used the Adam optimizer with a learning rate of 0.001. In regard to operations done within each forward pass, the pipeline goes as follows:

1. First take the entire input (list of all tokenized prompts) and grab the embeddings for each token from the pretrained embeddings model (FastText).
2. Reshape those embeddings to fit Pytorch's conv1D dimension requirements.
3. Run the convolution through the mentioned Pytorch function for each filter size.
4. Apply ReLU, removing all negative values
5. Apply Max-pooling, taking the strongest feeling/semantic from the feature map for that filter.
6. Concatenate the results from each filter (300 filters, so a vector that is 300 long)
7. We drop a few of these (half of them turn to 0 because of our dropout = 0.5)
8. Then the Pytorch Linear function will decide which class to finally decide

After the forward pass, the model computes the loss, then performs backpropagation to adjust weights that caused the loss. After each training epoch, we measure our model's performance on the validation set. We trained the model for 10 epochs.

RNN Implementation

Our RNN model reads the text one token at a time and predicts the LLM from its final hidden state.

The embeddings are passed through a dropout of 0.5 into a single-layer LSTM with 128 hidden units. An LSTM was chosen because our responses in our dataset are long, and LSTM helps by preserving information over long sequences. The output is a linear layer that maps the 128-dimensional final hidden state to the four LLM classes.

We trained this implementation with a cross-entropy loss and the Adam optimizer of a learning rate of 0.001 with a batch size of 32 for up to 12 epochs. After each epoch was completed, we computed the weighted F1 on the validation set and saved the model whenever the model has improved. We also added early stopping for this implementation if validation F1 did not improve for 3 epochs. The best model was evaluated on the test set.

To keep the CNN and RNN comparable, both use the same data splits, NLTK tokenizer, fastText embeddings, 256 token cap, and batch size.

CNN-RNN Hybrid Implementation

We combined the two models in 3 ways. They all use the same preprocessing to reduce the variety. Every model uses the NLTK tokenizer, a 256-token cut, and pretrained 300-dimensional fastText vectors that are fine-tuned during training.

Stacked CNN → RNN – Running the CNN first then feeding it into the RNN gives us a mixture of both where the CNN detects the short phrases and the formatting while the LSTM reads those patterns in order. The whole model is trained as one network, so the CNN's filters are updated by the LSTM's errors. The LSTM's final hidden state goes through dropout and a linear layer to give four class scores.

Parallel – A CNN branch and LSTM branch read the same embedding side by side. The CNN max pools its feature map over time while the LSTM outputs the final hidden state. The branches meet at the last layer, and the model is trained jointly.

Separate + soft vote – The CNN and RNN are trained separately, and their results are averaged together with equal weights. The class with the highest average probability is chosen to be the prediction. These models never get influenced by each other.

Training – To make the comparison fair, all the models were trained on the same training loop. Using the Adam optimizer with a learning rate of 0.001, with a batch size of 32, gradient clipping, and up to 12 epochs. The training stops if the validation hasn't improved for 3 epochs, resetting to the best validation weights. Because of the shared loop, the standalone CNN was trained with Adam with a learning rate of .001, instead of the Adadelta setting. All models use the same splits, and each model was run with 5 random seeds, changing the weight, initialization, and batch order.

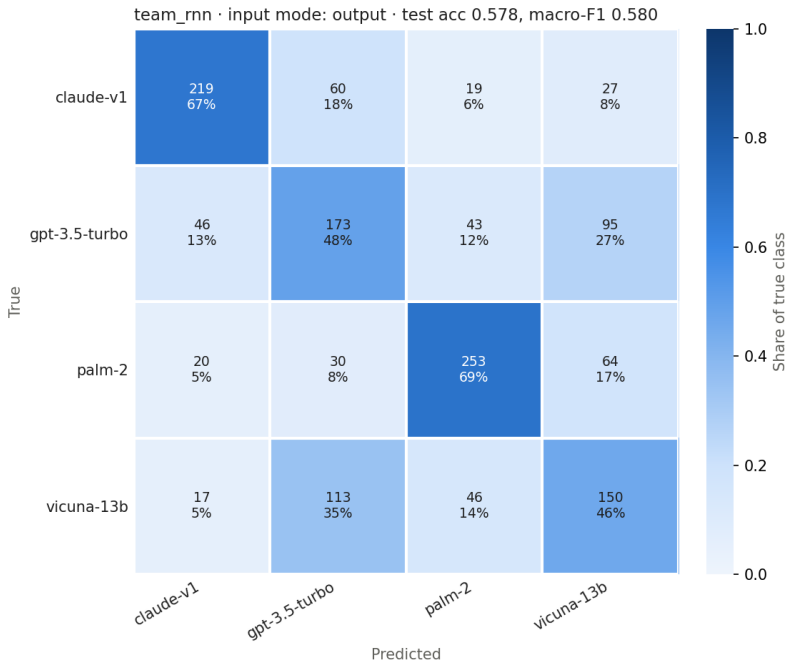
Results

Table 1: Test results for all models

Model	Val macro-F1	Test accuracy	Test macro-F1
CNN	0.671 ± 0.007	0.680 ± 0.005	0.681 ± 0.006
Soft-vote (CNN + RNN)	0.655 ± 0.017	0.659 ± 0.027	0.658 ± 0.028
Parallel (CNN ‖ RNN)	0.647 ± 0.008	0.647 ± 0.012	0.649 ± 0.007
RNN	0.590 ± 0.008	0.576 ± 0.004	0.576 ± 0.005
Stacked (CNN → RNN)	0.556 ± 0.009	0.557 ± 0.015	0.555 ± 0.018

Every model scored well above random guessing of 0.25. The CNN did the best with a 0.681 test macro-F1, while RNN reached 0.576.

Figure 1: RNN confusion matrix for the test set



This is an RNN confusion matrix on the test set with only response only. PaLM-2 has a 69% recall and Claude-v1 got 67%. These two models were identified the most often, while

GPT-3.5 with a recall of 48% and Vicuna-13B with a recall of 46% were mostly confused with each other.

Figure 2: CNN training log of loss and validation accuracy per epoch

```
 Epoch   |  Train Loss   |  Val Loss   |  Val Acc   |  Elapsed
-------------------------------------------------------------
   1     |   1.160825    |  0.955827   |   56.31    |   7.67
   2     |   0.803218    |  0.790316   |   64.25    |   5.71
   3     |   0.598929    |  0.724595   |   67.90    |   5.75
   4     |   0.388706    |  0.729460   |   67.06    |   5.97
   5     |   0.229679    |  0.726565   |   68.25    |   5.82
   6     |   0.130786    |  0.759384   |   68.25    |   5.77
   7     |   0.081279    |  0.781019   |   67.69    |   6.22
   8     |   0.059137    |  0.809283   |   67.79    |   6.09
   9     |   0.039695    |  0.845088   |   67.27    |   5.82
  10     |   0.034863    |  0.865808   |   67.90    |   5.83


Training complete! Best accuracy: 68.25%.
Validation loss: 0.8658
Validation accuracy: 67.90%
Test loss: 0.8834
Test accuracy: 67.43%
```

The stacked model was the worst, which is likely caused by a bottleneck being created when the LSTM reads the CNN's feature map.

In our setup, combining the models did not improve the standalone CNN. The ranking followed the strength of components, since the CNN was the strongest part, and the hybrids that retained most of the CNN did better. Which is probably because the CNN already captured the main signals, which are phrase and formatting patterns.

Since the test set has 1,375 responses, differences of about 2 points or less should be treated as noise. Additionally, the 5 seeds only changed the initialization and the batch order, not the data split, nor the model's hyperparameters.

Table 2: RQ2: F1 scores by input type for prompt only, response only and prompt + response

Input	CNN F1	RNN F1
Prompt only	0.2565	0.250
Response only	0.6844	0.562
Prompt + Response	0.6457	0.549

These are the F1 results of RNN and CNN from a single run.

Table 3: RQ3: Overall F1 when the test task was seen vs. unseen

Test Task	Seen F1	Unseen F1	Drop
Coding	0.551	0.432	0.119
Writing	0.527	0.449	0.078

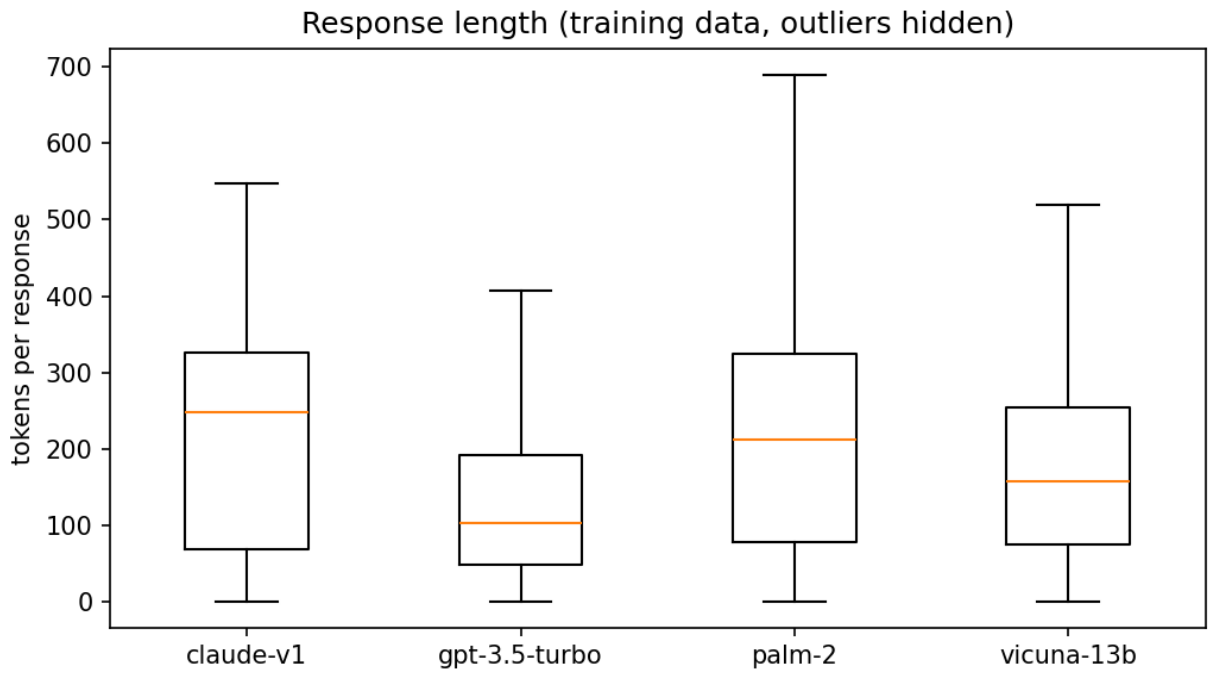
Table 4: RQ3: F1 score per LLM when the test task was seen vs. unseen

LLM	Coding Seen	Coding Unseen	Coding Change	Writing Seen	Writing Unseen	Writing Change
Claude-v1	0.752	0.617	-0.135	0.583	0.486	-0.097
GPT-3.5	0.509	0.364	-0.145	0.419	0.381	-0.038
PaLM-2	0.500	0.500	0.000	0.646	0.593	-0.053
Vicuna-13B	0.416	0.222	-0.194	0.446	0.327	-0.119

The Arena dataset has no task labels, so we labeled each prompt by task with the use of AI assistance and tested two tasks: coding and writing. For each task, the RNN was trained twice with the same amount of training data, one with tasks included, and one with it removed from training. Both of these runs were tested on the same examples.

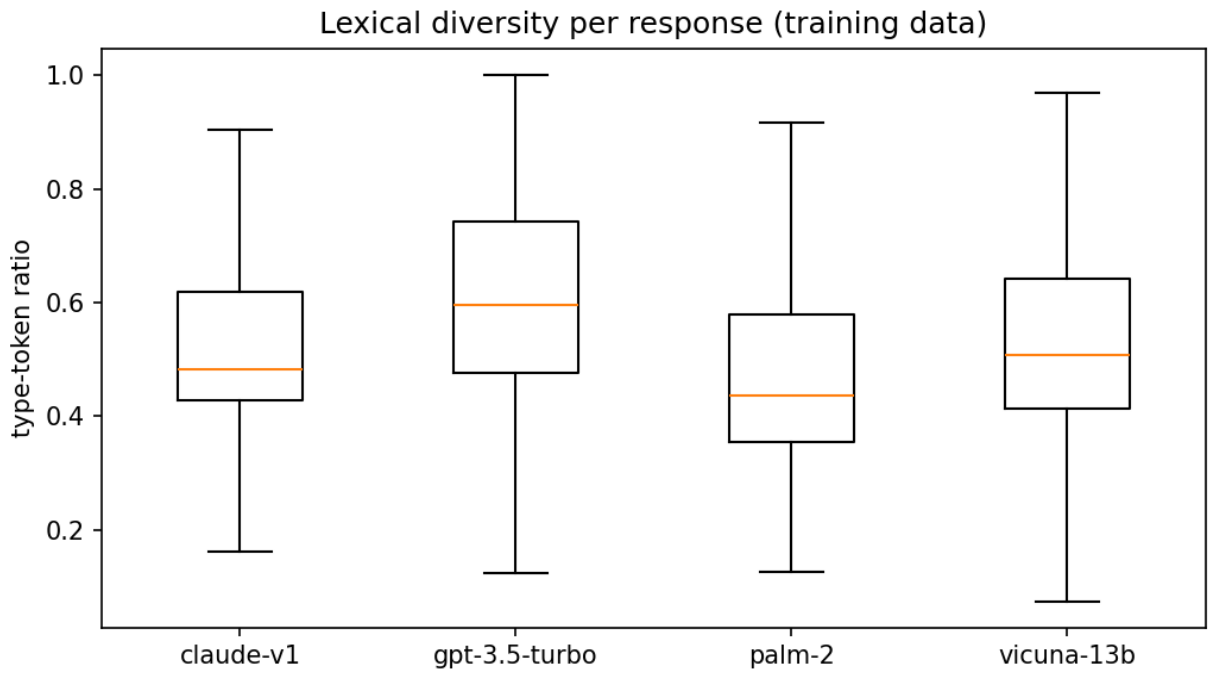
Figure 3: Response length per LLM

We computed the statistics on the training response for each LLM, like length in characters and tokens, vocabulary size, type-token ratio, and formatting counts.



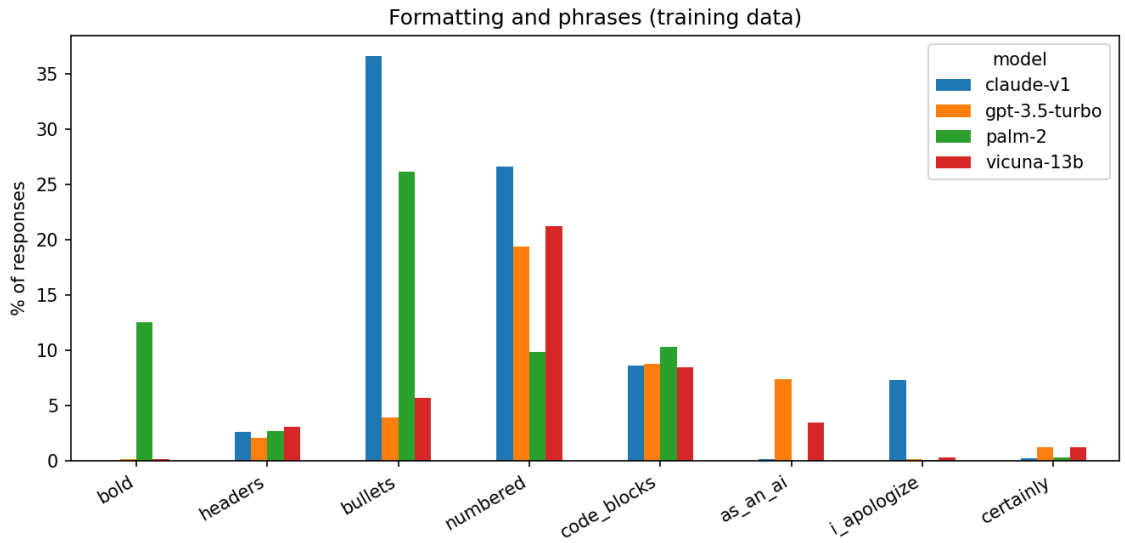
Length – The length follows this order: Claude-v1 (~250 tokens) → PaLM-2 (~210 tokens) → Vicuna-13b (~160 tokens) → gpt-3.5-turbo (~100 tokens). PaLM-2 had the widest spread, with its upper whisker reaching 600 tokens.

Figure 4: Type-token ratio per LLM



Lexical diversity - GPT-3.5-turbo has the highest type-token ratio (median about 0.6), while PaLM-2 has the lowest (about 0.44). Claude-v1 (about 0.48) and Vicuna-13B (about 0.51) fall in between. Type-token ratio drops as text gets longer, because words repeat more, so this difference is partly a length effect. GPT-3.5's high score may reflect its short responses rather than a richer vocabulary.

Figure 5: Formatting and phrase usage per LLM



Formatting:

- Almost only PaLM-2 (about 12.5% of its responses) had bolding; the other three are near 0%.
- Claude-v1 (36.6%) and PaLM-2 (26.2%) of bullet point usage, far more than GPT-3.5-turbo (about 4%) and Vicuna-13B (about 6%).
- For numbered lists, Claude-v1 (about 27%), Vicuna-13B (about 21%), and GPT-3.5-turbo (about 19%) use them more than PaLM-2 (about 10%).
- Phrases: “As an AI” appears mostly in GPT-3.5-turbo (7.4%) and Vicuna-13B (about 3.5%), and “I apologize” appears mostly in Claude-v1 (7.3%).
- Not distinguishing headers (about 2 to 3% for all) and code blocks (about 8 to 10% for all) look similar across models, so they carry little signal. “Certainly” is rare for every model (about 1% or less).

In depth analysis/Experiments

CNN Conclusion

Through 10 epochs of training the CNN model, we found that it started off with an accuracy of around 56%, but as it continued to train, it quickly shot up to the 67% ranges, then hovering around that range, with 67-68% being the highest validation accuracy seen.

The training and validation loss held even at the first couple of epochs, decreasing at a good rate, but very quickly did it show signs of overfitting, as training loss plummeted while validation loss kept even and even increased.

One thing that we saw was that while evaluating the test set, its accuracy seemed to be consistently a bit lower than the validation accuracy, which suggests that it picked up some generalization, being able to predict almost as well on completely new data compared to the validation set. The lower percentage points also hint at potentially a bit of overfitting the validation, but it's not an issue in this case.

RNN Conclusion

Looking at the RNN's confusion matrix, the most common mistake was confusing GPT-3.5 and Vicuna-13B as 36% of all mistakes were between these two models. These two models both write in a very similar style as phrases like "As an AI…" and "I'm sorry, …" appear almost only in these two models and they use formatting nearly the same as there were numbered lists in 19% vs. 21% responses and rarely use bolded text. This is likely due to Vicuna being a model that was fine-tuned on ShareGPT conversations on ChatGPT, so it learned to imitate GPT-3.5's style. PaLM-2 was the easiest to identify with a recall of 69%, which was identified with its heavy use of bold words. Claude-v1 was the second close to identify with a recall of 67% as it has distinctive habits. More than a third of the responses started with "Here is…" or "Here are…" and also says "I apologize" and uses bullet points as well.

For our LSTM implementation, on its own the prompt gives the LSTM almost no information about which model answered. Prompt-only input has achieved a weighted F1 score of 0.250 with an accuracy of 0.266. This is because the prompt is written by the user, not the LLM, and all four models answer similar Chatbot Arena prompts. In addition, adding the prompt to the response did not help as F1 fell slightly, from 0.562 for response only to 0.549 for prompt and response, a change of 0.013. The likely cause of this is how the input was built as the LSTM reads at most 256 tokens, and the prompt takes up to 64 of them. In 35% of the test dataset examples, this pushed part of the response past its limit, so the

model sees less of the LLM-written text, which is where the stylistic signal is. In addition, the LSTM predicts its final hidden state, and in the sequence, the first thing that comes is the prompt, so it has very little influence on the prediction.

Hybrid Conclusion

None of the combination models beat the standalone model, but the soft-vote and parallel models are much higher than the standalone RNN. So, combining helped the weaker model without surpassing the stronger one. The gap between soft and parallel is within standard deviation, so it's hard to say which is better.

RQ1 Conclusions

Based on the performance of our best model, the CNN with an f1 score of 0.681, it can be concluded that you **can predict the LLM based upon the response given**. The model essentially gets it correct 2/3 of the time, which is almost 2.5 times better than if you were to guess randomly with a 1/4 chance.

RQ2 Conclusions

As expected, the F1 score for prompt only with our two models is around 25%, the same as randomly guessing which class to classify out of our 4 classes. **In other words, you cannot predict which LLM it is based on solely the prompt.** A pattern we can see is the F1 score for CNN being higher than RNN across all categories. More importantly, there's a pattern of having the Prompt + Response decreasing the F1 score for both classifiers. **This suggests that the prompt doesn't help guessing which model it is, and that adding it can potentially introduce noise that reduces performance.**

RQ3 Conclusions

Comparing the results after running the two tasks on coding and writing, performance dropped on both unseen tasks, more for coding than writing. This is likely due to responses that look very different from a normal chat. PaLM-2 stayed the most consistent, due to its most identifiable style being bold formatting. Vicuna dropped the most in performance as its style overlaps with GPT-3.5's, so the **classifier learns part of a general fingerprint, but much of it is task specific.**

Lessons Learned

1. When working on the arena dataset, the same prompt is often answered by two different models. If we had split the data by row, the same prompt could appear in both training and testing, so we split by prompt instead.
2. When we first ran the RNN, it made its prediction from the last position, which for short responses, there was padding instead of text. To solve this, packing the sequences fixed it, so the prediction comes from the last token that is available for the text.
3. When initially running the RQ3, we first ran into an issue when the data was labeled. There was a change in how the lines ended, which made machine labels break. After fixing this issue, the measured drop on unseen tasks grew significantly.
4. The LLMs vary in how long they write, how much they vary in their words, and how they format their answers. Bold and bullet lists are clear structural signals, and “As an AI” and “I apologize” are the clear phrase signals.

Experiences Gained

Having seen classifier implementations in the past without the understanding behind all of the parts that go into making it work, it’s satisfying to not just learn about everything that goes into it (tokenizers, embeddings, activation functions, optimizers, hyperparameters, etc.), but to actually get practical experience in implementing it.